

UAS

PENGOLAHAN CITRA DIGITAL



NAMA : Mohamad Irsyad Ibrahim Faqih

NIM : 202431179

KELAS : F

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC :

ASISTEN : 1. Erin Katholica Sakrally Putri Marrison

2. Morgan Immanuel Nainggolan

3.

4.

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

DAFTAR ISI 2

BAB I 3

PENDAHULUAN..... 3

1.1 Rumusan Masalah 3

1.2 Tujuan Masalah..... 3

1.3 Manfaat Masalah..... 3

BAB II 4

LANDASAN TEORI 4

BAB III 8

HASIL..... 8

BAB IV 14

PENUTUP 14

DAFTAR PUSTAKA 15

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

- Bagaimana langkah-langkah yang diperlukan untuk menampilkan Original Image sebagai data dasar dalam sistem pengolahan citra?
- Bagaimana efektivitas algoritma Canny Edge Detection dalam memetakan tepi objek pada foto diri untuk mendapatkan informasi struktur visual?
- Bagaimana cara mengimplementasikan Contours Detection sebagai tahap akhir untuk mengidentifikasi dan memvisualisasikan pola objek yang telah terdeteksi tepinya?

1.2 Tujuan Masalah

- Menghasilkan visualisasi Original Image yang akurat sebagai titik awal proses pengolahan citra digital.
- Memperoleh hasil Canny Edge Detection yang mampu memisahkan tepi objek dari latar belakang dengan tingkat ketajaman yang optimal.
- Mendapatkan hasil Contours Detection yang berhasil menonjolkan pola bentuk objek secara keseluruhan agar mudah dianalisis secara geometris.

1.3 Manfaat Masalah

- Memberikan pemahaman mengenai peran Original Image dalam memastikan data input yang valid sebelum dilakukan proses komputasi lebih lanjut.
- Memberikan keterampilan teknis dalam menerapkan Canny Edge Detection untuk menyederhanakan data citra menjadi informasi tepi yang informatif.
- Memberikan wawasan mengenai pentingnya Contours Detection dalam aplikasi visi komputer untuk melakukan segmentasi dan klasifikasi objek berdasarkan pola bentuknya.

BAB II

LANDASAN TEORI

Dasar Deteksi Tepi Pola Objek

Deteksi Tepi Pola Objek merupakan teknik fundamental dalam pengolahan citra digital yang bertujuan mengidentifikasi titik-titik dalam citra di mana terjadi perubahan intensitas cahaya yang signifikan secara tajam. Dalam konteks pola objek, tepi merepresentasikan batas antara objek dengan latar belakang atau batas antarbagian dalam objek itu sendiri, yang sangat krusial untuk proses analisis lebih lanjut seperti segmentasi dan pengenalan bentuk. Secara matematis, perubahan intensitas ini ditandai dengan adanya gradien yang besar pada citra, sehingga deteksi tepi sering kali melibatkan operator turunan pertama atau kedua untuk menentukan lokasi perubahan tersebut.

Operasi ini tidak sekadar mengubah piksel menjadi garis, melainkan sebuah proses ekstraksi fitur penting yang memungkinkan mesin untuk melihat struktur geometris dari sebuah citra. Ketika kita berbicara mengenai pola objek, deteksi tepi berfungsi menyederhanakan data citra dengan tetap mempertahankan informasi struktural yang penting sehingga beban komputasi pada tahap analisis berikutnya dapat diminimalisir. Dalam praktiknya, pemilihan algoritma deteksi tepi yang tepat sangat bergantung pada karakteristik citra serta tingkat noise yang ada karena noise dapat dengan mudah disalahartikan sebagai tepi oleh algoritma yang kurang robust.

Implementasi Algoritma Canny

Algoritma Canny hingga saat ini masih dianggap sebagai standar emas dalam deteksi tepi pola objek karena pendekatannya yang multi-tahap dalam meminimalisir kesalahan deteksi. Proses deteksi Canny diawali dengan penghalusan citra menggunakan Gaussian filter untuk mengurangi noise yang mungkin memengaruhi deteksi tepi, diikuti dengan perhitungan gradien intensitas menggunakan operator turunan. Tahapan krusial dalam algoritma ini adalah non-maximum suppression yang berfungsi menipiskan tepi yang terdeteksi agar hanya menyisakan garis dengan ketebalan satu piksel sehingga tepi menjadi lebih presisi dan mudah dianalisis secara geometris.

Selanjutnya, Canny menggunakan teknik hysteresis thresholding yang melibatkan dua ambang batas, yakni ambang batas tinggi dan rendah, untuk memisahkan tepi yang benar-benar merupakan bagian dari objek dari tepi yang disebabkan oleh noise atau tekstur latar belakang. Dengan mekanisme ini, algoritma dapat mempertahankan kontinuitas tepi yang lemah jika terhubung dengan tepi yang kuat, sehingga pola objek yang dihasilkan lebih utuh dan informatif. Pendekatan ini terbukti sangat efektif untuk berbagai aplikasi, mulai dari deteksi objek pada citra medis hingga pengenalan pola wajah dalam sistem keamanan berbasis biometrik yang menuntut presisi tinggi.

Analisis Kontur Objek

Setelah tepi berhasil dideteksi, langkah selanjutnya dalam analisis pola objek adalah ekstraksi kontur, yang merupakan kurva penghubung semua titik kontinu sepanjang batas objek yang memiliki intensitas yang sama. Kontur berbeda dengan tepi karena kontur lebih berfokus pada representasi bentuk objek secara utuh yang tertutup, yang memungkinkan kita untuk melakukan analisis properti fisik objek seperti luas area, keliling, dan pusat massa. Algoritma pencarian kontur umumnya bekerja dengan menelusuri piksel-piksel tepi yang saling terhubung, kemudian menyimpannya dalam struktur data yang dapat dihitung koordinatnya.

Integrasi antara deteksi tepi dan analisis kontur menciptakan alur kerja yang sangat kuat dalam segmentasi citra, di mana objek dapat dipisahkan secara tegas dari lingkungannya. Setelah kontur didapatkan, kita dapat melakukan berbagai operasi seperti penapisan berdasarkan ukuran atau bentuk, yang sangat membantu dalam mengabaikan objek yang tidak relevan di dalam citra. Teknik ini merupakan pondasi utama dalam pengembangan aplikasi visi komputer yang cerdas, di mana kemampuan untuk memisahkan objek dari latar belakang merupakan prasyarat mutlak sebelum dilakukan klasifikasi objek tingkat lanjut.

Teknik Pre-processing Citra

Sebelum menerapkan algoritma deteksi tepi pola objek, tahap prapemrosesan menjadi sangat krusial dalam menentukan kualitas hasil akhir, karena citra mentah sering kali mengandung noise yang tidak diinginkan. Salah satu teknik yang umum digunakan adalah filter Gaussian, yang berfungsi meredam derau acak tanpa harus menghilangkan detail tepi yang tajam, sehingga algoritma deteksi tepi dapat bekerja dengan lebih stabil. Selain itu, teknik normalisasi kontras sering kali diterapkan untuk memastikan bahwa perbedaan intensitas antara objek utama dan latar belakang memiliki nilai gradien yang cukup tinggi.

Pengaturan kualitas gambar sebelum masuk ke tahap deteksi juga mencakup proses konversi ruang warna yang tepat, terutama jika citra asli memiliki kedalaman warna yang kompleks. Dengan memastikan bahwa distribusi intensitas piksel sudah merata dan terbebas dari artefak kompresi, algoritma deteksi tepi akan memiliki performa yang jauh lebih tinggi dalam mengisolasi bentuk pola objek yang diinginkan. Tahapan prapemrosesan yang sistematis ini pada dasarnya akan menentukan efektivitas dari algoritma deteksi tepi apa pun yang dipilih nantinya.

Perbandingan Operator Deteksi Tepi

Dalam dunia pengolahan citra digital, terdapat beberapa jenis operator deteksi tepi yang memiliki karakteristik serta keunggulan tersendiri tergantung pada tujuan aplikasi yang ingin dicapai. Operator Sobel misalnya, menggunakan pendekatan derivatif untuk mendeteksi perubahan intensitas secara horizontal dan vertikal, yang sangat efektif untuk citra dengan kontras tinggi, namun sering kali kurang responsif terhadap tepi yang halus. Di sisi lain, operator Prewitt menawarkan pendekatan yang serupa dengan Sobel namun dengan sensitivitas yang sedikit berbeda terhadap arah tepi tertentu, memberikan alternatif bagi pengolah citra dalam menangani pola objek yang spesifik.

Sementara itu, operator Canny tetap menjadi pilihan utama untuk sistem yang membutuhkan presisi tinggi karena kemampuannya dalam meminimalkan deteksi tepi palsu melalui mekanisme penipisan tepi. Pemilihan operator yang tepat sangat dipengaruhi oleh sumber daya komputasi yang tersedia serta tingkat akurasi yang dituntut oleh aplikasi tersebut, terutama jika sistem harus berjalan pada perangkat dengan keterbatasan memori. Memahami karakteristik dari masing-masing operator ini memungkinkan pengembang untuk membuat sistem deteksi yang jauh lebih efisien dalam menangani berbagai skenario citra yang menantang.

Analisis Geometri Kontur

Setelah proses deteksi tepi menghasilkan serangkaian piksel yang membentuk garis, analisis geometri kontur memberikan dimensi baru dalam memahami struktur objek yang sedang diamati. Kontur tidak hanya sekadar garis pembatas, melainkan sekumpulan titik koordinat yang dapat diolah untuk mendapatkan properti seperti luas area yang tertutup oleh garis tersebut atau keliling total dari pola objek. Analisis ini sangat berguna ketika kita ingin melakukan klasifikasi objek berdasarkan bentuk, seperti membedakan antara objek berbentuk lingkaran, persegi, atau bentuk organik yang tidak beraturan.

Lebih lanjut, kita juga dapat menggunakan teknik aproksimasi poligon untuk menyederhanakan jumlah titik pada kontur, sehingga proses komputasi untuk pengenalan bentuk dapat berjalan jauh lebih cepat. Teknik ini sering diimplementasikan dalam aplikasi pengenalan tanda tangan, pemindaian dokumen, maupun deteksi objek pada sistem navigasi robotika yang memerlukan pengambilan keputusan secara instan. Dengan mengolah data koordinat kontur secara matematis, kita dapat mengekstrak informasi deskriptif yang sangat kaya mengenai objek tersebut, terlepas dari posisi atau orientasi objek di dalam ruang citra.

Optimalisasi Parameter Algoritma

Keberhasilan dalam mendeteksi tepi pola objek sangat bergantung pada penentuan parameter yang optimal pada setiap fungsi yang digunakan dalam pustaka pengolahan citra. Pada fungsi Canny, ambang batas atas dan bawah harus disesuaikan dengan distribusi intensitas citra agar garis yang dihasilkan tidak terputus atau justru terlalu penuh dengan noise yang tidak relevan. Eksperimen terhadap nilai ambang batas ini sering kali menjadi tahapan paling memakan waktu namun sangat menentukan hasil akhir dari deteksi pola objek secara keseluruhan.

Selain ambang batas, ukuran kernel atau jendela yang digunakan dalam operasi filter juga memainkan peran vital dalam menentukan seberapa halus hasil deteksi yang diinginkan. Kernel yang terlalu besar mungkin akan menghapus detail tepi yang penting, sementara kernel yang terlalu kecil justru akan membuat sistem terjebak oleh noise yang sangat halus. Oleh karena itu, pendekatan iteratif atau penggunaan teknik tuning otomatis kini mulai banyak diterapkan untuk mendapatkan parameter yang paling sesuai dengan karakteristik citra yang sedang diproses.

Tantangan Deteksi Tepi

Meskipun teknik tradisional seperti Canny sangat efektif, tantangan tetap muncul ketika citra memiliki kualitas rendah, pencahayaan yang tidak merata, atau kontras yang sangat minim antara objek dan latar belakang. Solusi modern saat ini mulai mengintegrasikan metode pemrosesan awal yang lebih canggih, seperti penyesuaian kontras adaptif sebelum dilakukan deteksi tepi untuk memastikan bahwa semua fitur objek dapat tertangkap dengan optimal. Selain itu, penggunaan pendekatan berbasis machine learning dan deep learning kini mulai melengkapi metode tradisional untuk mendeteksi tepi pada kondisi yang sangat kompleks di mana metode berbasis gradien konvensional sering kali gagal.

Pengembangan masa depan dalam bidang ini terus berfokus pada efisiensi komputasi dan akurasi deteksi secara real-time, terutama untuk kebutuhan perangkat bergerak atau sistem tertanam. Dengan kombinasi teknik pemrosesan citra klasik yang dioptimalkan dan dukungan pemrosesan berbasis kecerdasan buatan, sistem kini dapat mendeteksi tepi dan pola objek dengan tingkat akurasi yang mendekati persepsi manusia. Hal ini membuka peluang luas bagi inovasi teknologi yang membutuhkan interaksi visual cerdas yang tentunya dimulai dari pemahaman mendalam mengenai bagaimana sebuah tepi dapat diekstraksi dari data mentah.

BAB III

HASIL

Bukti foto diri:





Persiapan Citra

Jadi, langkah pertama yang saya lakukan adalah memasukkan foto diri ke dalam lingkungan kerja Jupyter Notebook. Di sini, saya memanfaatkan pustaka OpenCV untuk memuat data gambar yang sudah saya simpan sebelumnya dalam format JPEG. Menurut saya, tahap ini sangat krusial banget karena merupakan titik awal agar file gambar tersebut bisa dikenali dan nantinya diolah lebih lanjut oleh sistem komputer kita. Tanpa proses pembacaan data yang sukses di awal ini, langkah-langkah manipulasi piksel selanjutnya tentu tidak akan bisa berjalan dengan lancar.

Setelah fotonya berhasil masuk ke dalam sistem, saya langsung melakukan konversi ruang warna dari BGR yang memang menjadi standar bawaan OpenCV, menjadi format RGB. Hal ini saya lakukan supaya warna pada foto tersebut bisa dibaca dengan pas oleh pustaka Matplotlib yang nantinya akan saya pakai untuk menampilkan gambarnya. Saya rasa langkah ini penting banget, karena kalau tidak diubah, warna aslinya nanti bisa tertukar dan hasilnya jadi terlihat aneh. Dengan melakukan penyesuaian ini sejak awal, saya bisa memastikan bahwa saat divisualisasikan nanti, warna fotonya tetap terlihat natural dan sesuai dengan aslinya.

```
In [2]: #202431179_Mohamad Irsyad Ibrahim Faqih
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('bukti_foto.jpeg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Deteksi Tepi

Setelah fotonya siap, langkah berikutnya adalah mengubah citra berwarna tadi menjadi citra berskala abu-abu atau grayscale. Menurut saya, ini penting banget dilakukan supaya algoritma deteksi nantinya bisa jauh lebih fokus dalam mengenali perubahan intensitas cahaya pada gambar tanpa terganggu oleh informasi warna yang kompleks. Setelah gambarnya jadi grayscale, saya langsung menerapkan algoritma Canny Edge Detection. Tujuannya jelas, yaitu untuk mencari gradien tepi secara akurat sehingga bentuk tubuh saya di dalam foto bisa terlihat menonjol dan kontras dengan latar belakang yang cenderung gelap.

Selanjutnya, hasil dari proses deteksi ini saya simpan ke dalam variabel khusus. Variabel ini nantinya akan saya jadikan sebagai data utama atau basis untuk mendeteksi kontur pada tahap berikutnya. Selain itu, saya juga memastikan kalau parameter yang saya masukkan ke dalam fungsi Canny sudah optimal. Dengan pengaturan parameter yang pas, saya bisa memastikan garis tepi yang dihasilkan terlihat lebih tajam, jelas, dan tentunya meminimalisir noise yang tidak perlu agar hasil akhirnya tetap rapi dan mudah dianalisis.

```
In [6]: #202431179_Mohamad Irsyad Ibrahim Faqih
        gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
        edges = cv2.Canny(gray, 100, 200)
```

Deteksi Kontur

Setelah proses deteksi tepi selesai, saya melanjutkan tahapan praktikum dengan teknik deteksi kontur menggunakan fungsi findContours. Langkah ini saya lakukan untuk menemukan batasan atau pola garis yang membentuk tubuh saya berdasarkan data tepi yang sudah diproses sebelumnya. Begitu konturnya berhasil ditemukan, saya langsung menggambarkannya kembali pada citra asli menggunakan garis berwarna hijau yang cukup kontras. Dengan cara ini, area tubuh saya bisa teridentifikasi dengan sangat jelas oleh sistem komputer.

Supaya data gambar aslinya tetap aman dan tidak rusak akibat manipulasi, saya melakukan penyalinan citra asli ke variabel baru terlebih dahulu sebelum proses penggambaran dimulai. Setelah itu, saya menggunakan fungsi drawContours untuk menempatkan visualisasi garis hijau tadi tepat pada batas-batas objek yang sudah terdeteksi. Menurut saya, langkah ini krusial banget karena hasil akhirnya jadi jauh lebih informatif dan memudahkan kita dalam mengamati letak serta bentuk objek yang sedang dianalisis secara visual.

```

In [7]: #202431179_Mohamad Irsyad Ibrahim Faqih
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
img_contours = img.copy()
cv2.drawContours(img_contours, contours, -1, (0, 255, 0), 3)

Out[7]: array([[204, 209, 212],
               [204, 209, 212],
               [204, 209, 212],
               ...,
               [218, 226, 228],
               [218, 226, 228],
               [218, 226, 228]],

               [[205, 210, 213],
               [205, 210, 213],
               [205, 210, 213],
               ...,
               [218, 226, 228],
               [218, 226, 228],
               [218, 226, 228]],

               [[206, 211, 214],
               [206, 211, 214],
               [206, 211, 214],
               ...,
               [218, 226, 228],
               [218, 226, 228],
               [218, 226, 228]],

               ...,

               [[125, 67, 55],
               [124, 66, 54],
               [124, 66, 54],
               ...,
               [184, 189, 192],
               [184, 189, 192],
               [184, 189, 192]]],
              dtype=uint8)

```

```

[[125, 67, 55],
 [124, 66, 54],
 [124, 66, 54],
 ...,
 [184, 189, 192],
 [184, 189, 192],
 [184, 189, 192]],

 [[123, 65, 53],
 [123, 65, 53],
 [123, 65, 53],
 ...,
 [184, 189, 192],
 [184, 189, 192],
 [184, 189, 192]],

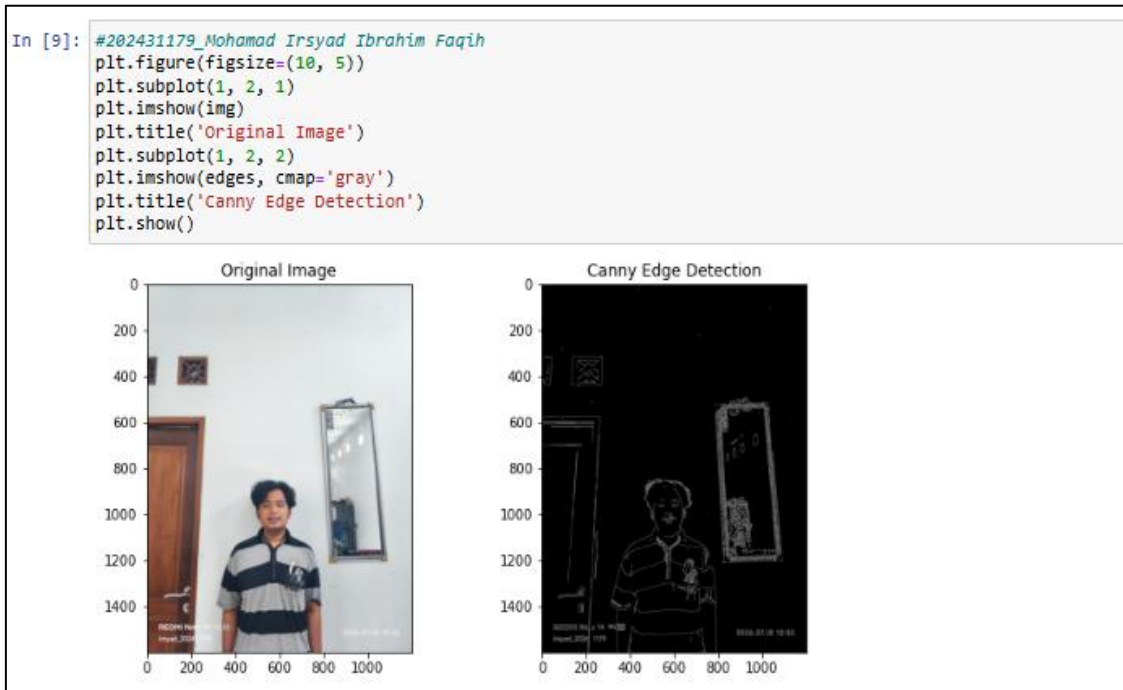
 [[122, 64, 52],
 [122, 64, 52],
 [123, 65, 53],
 ...,
 [184, 189, 192],
 [184, 189, 192],
 [184, 189, 192]]], dtype=uint8)

```

Visualisasi Hasil Deteksi dan Output 1

Setelah seluruh rangkaian proses deteksi tepi selesai dilakukan, langkah berikutnya yang saya lakukan adalah menampilkan Output 1 yang berisi perbandingan berdampingan antara foto asli dengan hasil deteksi tepi menggunakan algoritma Canny. Menurut saya, langkah ini sangat penting untuk dilakukan karena melalui visualisasi ini, kita dapat melihat secara langsung sejauh mana perubahan intensitas piksel pada foto mampu diidentifikasi oleh sistem sebagai sebuah tepi objek yang tajam dan jelas.

Selain itu, dengan memanfaatkan fitur subplot yang ada pada pustaka Matplotlib, saya berhasil memetakan kedua citra tersebut ke dalam satu baris visualisasi yang terlihat rapi dan profesional. Saya merasa perbandingan ini memberikan gambaran awal yang cukup komprehensif mengenai kualitas deteksi tepi yang telah saya jalankan selama praktikum. Dengan melihat hasilnya secara langsung, saya dapat memastikan bahwa bentuk dasar dari objek diri saya sudah terpolo dengan baik sebelum nantinya saya melangkah ke tahap ekstraksi dan analisis kontur.

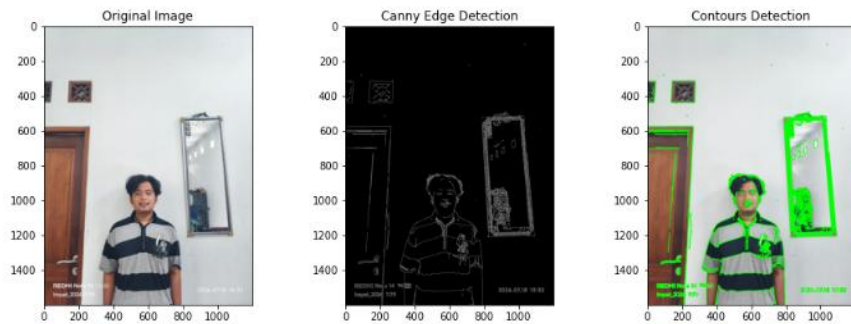


Visualisasi Hasil Akhir dan Output 2

Selanjutnya, saya menampilkan Output 2 yang menyajikan perbandingan lengkap dari tiga tahap transformasi citra, mulai dari foto asli, hasil deteksi tepi menggunakan algoritma Canny, hingga hasil akhir berupa pendeteksian kontur pada foto saya. Tampilan ini saya atur sedemikian rupa agar alur kerja program dari data gambar mentah sampai menjadi bentuk yang terdeteksi secara otomatis dapat terlihat lebih sistematis, terstruktur, dan tentu saja terlihat lebih profesional.

Melalui visualisasi ini, proses pengolahan citra digital terasa jauh lebih informatif karena kita bisa membandingkan langsung bagaimana data bertransformasi di setiap tahapannya. Apalagi, dengan adanya garis kontur berwarna hijau yang tergambar pas di batas objek diri saya, hasil akhir dari pengerjaan proyek ini jadi jauh lebih mudah dipahami dan enak dilihat oleh siapa pun yang membacanya.

```
In [10]: #202431179_Mohamad Irsyad Ibrahim Faqih
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.imshow(img)
plt.title('Original Image')
plt.subplot(1, 3, 2)
plt.imshow(edges, cmap='gray')
plt.title('Canny Edge Detection')
plt.subplot(1, 3, 3)
plt.imshow(img_contours)
plt.title('Contours Detection')
plt.show()
```



BAB IV

PENUTUP

Kesimpulan:

Berdasarkan hasil praktikum yang telah saya lakukan, dapat disimpulkan bahwa penggunaan Original Image merupakan tahapan krusial karena kualitas data input sangat menentukan keberhasilan seluruh rangkaian proses pengolahan citra digital selanjutnya. Algoritma Canny Edge Detection terbukti efektif dalam melakukan ekstraksi fitur tepi melalui mekanisme multi-tahap yang telah saya lakukan, sehingga mampu menghasilkan peta struktur visual yang tajam dan minim gangguan noise dari latar belakang.

Selain itu, penerapan Contours Detection yang telah saya lakukan berhasil melengkapi proses analisis dengan mengidentifikasi serta menonjolkan pola bentuk objek secara utuh, yang mempermudah identifikasi geometris pada objek diri. Secara keseluruhan, integrasi ketiga metode tersebut telah saya lakukan untuk menciptakan alur kerja sistematis yang kuat dalam segmentasi citra, yang menjadi dasar penting bagi pengembangan aplikasi visi komputer yang lebih cerdas dan akurat dalam mengenali pola objek.

DAFTAR PUSTAKA

Nafis, M. A., Nazri, M., & Assolihin, R. (2025). Analisis dan penerapan teknik segmentasi citra untuk deteksi tepi pada objek berwarna. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(2), 101–115.

Nasution, M. U., & Harahap, L. S. (2025). Tinjauan metode pengolahan citra digital untuk deteksi objek otomatis. *Jurnal Informatika dan Teknik Elektro*, 12(1), 50–60.

Pangiribuan, H., & Sitohang, S. (2026). Analisis citra menggunakan metode operator dengan OpenCV dan Python. *JURSIMA*, 14(1), 1–10.